

Research Progress Report: Stateful Recursive Reasoning on CCB-Learn

D1 twelve-run confirmation

Pratham Lahoti

August 2026

Abstract

This report records the first controlled GPU study of recursive reasoning models on D1 of CCB-Learn, a locally implemented, leakage-audited version of the Complexity Ceiling Benchmark (CCB). We compare a direct Transformer, vanilla Tiny Recursive Model (TRM), Deep Improvement Supervision TRM (DIS-TRM), and our Stateful Transition Recursive Model (STRM), with three optimization seeds per model. The headline result is scientifically interesting but not yet a method claim: STRM is highly seed-sensitive. Its held-out-depth exact accuracies are 5.17%, 0.83%, and 99.17%; direct Transformer and vanilla TRM remain at 0% for every seed. Seed 2 must be independently audited and reproduced before it is interpreted as a generalization result.

1 Question, scope, and importance

The research question is whether a small weight-shared recursive neural model can learn a compositional transition rule at shallow depths and execute it at substantially greater unseen depths. This is a specialised but consequential algorithmic-generalization problem: success requires reusable state transformation rather than depth-specific pattern matching.

The project began from the instruction to apply TRM to CCB-style tasks. Gaussian-splatting object deletion was explicitly removed from scope. This report is solely about iterative/recurrent reasoning, neural state-transition execution, and compact models that trade parameter count for repeated computation.

2 Sources and prior work

- CCB paper: [arXiv](#); task code: [GitHub](#).
- TRM: [paper](#); code: [repository](#).
- DIS: [paper](#); code: [repository](#).
- Closest comparators: [Fast-Slow Latent Recurrence](#), [Looped Transformers](#), and [Universal Transformers](#).

Shared recursive updates, looped computation, and persistent latent state are not new individually. The candidate contribution is a precise stateful transition formulation plus controlled trace-level evaluation on CCB-Learn. No claim that STRM is the first recurrent or persistent-latent architecture is appropriate without a systematic literature and code audit.

3 What has been implemented

1. Deterministic D1, D2, and D3 generators; exact oracle traces; CCB serialization/parsing; and official-record compatibility checks.
2. Leakage-resistant deterministic splits, manifests, hashes, depth and structural-holdout infrastructure, and overlap audits.
3. Metrics for final/trace/transition exactness, retention p_d , horizon h_{50} , first divergence k^* , TFBC, depth AUC, and confidence intervals.
4. Direct Transformer, GRU/LSTM, looped Transformer, vanilla TRM, DIS-TRM, fast-slow recurrence, STRM, and a D3 GNN baseline.
5. CPU tests, tiny-data overfit tests, configuration/logging/checkpoint pipelines, and atomic Kaggle persistence.

Before model training, the local oracle exactly reproduced all 400 supplied official records for each of D1, D2, and D3. This establishes compatibility with those records but does not make CCB-Learn an identical reconstruction of unreleased upstream generators or tests.

4 Method and protocol

4.1 STRM

STRM carries an explicit answer-state estimate y and latent working state z across the operation sequence. For each operation it applies shared inner updates with recall of the initial state and current operation, then emits the next full structured state. Vanilla TRM supplies recursive refinement; STRM makes the transformed answer state persistent and explicit while retaining a latent workspace.

4.2 D1 study

Every run used width 64, four layers/loops, batch size 64, learning rate 10^{-3} , 10,000 steps, and 200 bootstrap resamples. Parameter counts are 248,649 for Transformer, 123,657 for TRM/DIS-TRM, and 153,033 for STRM. Official sealed evaluation was disabled.

The same deterministic splits were built once before the model-by-seed loop: train/validation depths 1–20; held-out test depths 25, 30, 35, 40, 45, and 50 with 100 examples each (600 total); and strong depths 60, 80, and 100 with 100 examples each (300 total). A seed therefore changes initialization and minibatch order, not the evaluation instances.

5 Metrics

Final exact accuracy requires the complete final structured state to be correct. Transition exact accuracy measures intermediate transition correctness. Retention p_d summarizes step-to-step survival, while h_{50} is the inferred depth at 50% success. Depth AUC summarizes performance across the held-out depths. These are intentionally complementary: a single early state error can make a complete long trajectory final-exact incorrect.

6 D1 results: all twelve runs

Table 1: Held-out-depth 25–50 results for every run. Final and transition are percentages. Strong final denotes final exact accuracy at depths 60–100.

Model	Seed	Final	Transition	p_d	h_{50}	AUC	Strong final
Transformer	0	0.00	11.19	0.500	1.00	0.000	0.00
Transformer	1	0.00	11.17	0.500	1.00	0.000	0.00
Transformer	2	0.00	11.26	0.500	1.00	0.000	0.00
TRM	0	0.00	9.68	0.500	1.00	0.000	0.00
TRM	1	0.00	10.51	0.500	1.00	0.000	0.00
TRM	2	0.00	2.30	0.500	1.00	0.000	0.00
DIS-TRM	0	0.33	27.80	0.843	4.06	0.004	0.00
DIS-TRM	1	0.50	29.78	0.851	4.28	0.004	0.00
DIS-TRM	2	0.17	26.97	0.814	3.37	0.001	0.00
STRM	0	5.17	46.15	0.916	7.88	0.045	0.00
STRM	1	0.83	30.54	0.863	4.71	0.008	0.00
STRM	2	99.17	99.69	1.000	3104.24	0.990	92.33

Table 2: Three-seed aggregation. STRM mean is dominated by its extreme seed-2 result, so the median and range are necessary.

Model	Mean final	Median final	Final range	Mean transition	Mean p_d	Mean strong
Transformer	0.00	0.00	0.00–0.00	11.21	0.500	0.00
TRM	0.00	0.00	0.00–0.00	7.49	0.500	0.00
DIS-TRM	0.33	0.33	0.17–0.50	28.19	0.836	0.00
STRM	35.06	5.17	0.83–99.17	58.79	0.926	30.78

7 Interpretation

Baseline evidence. Direct Transformer and vanilla TRM produce no complete held-out-depth solutions at this budget. DIS-TRM consistently improves local transition behavior (about 28% transition exactness and $p_d \approx 0.836$) but does not sustain complete long trajectories.

STRM signal. STRM seeds 0 and 1 partially outperform those baselines. Seed 2 is qualitatively different: 595/600 final held-out-depth states and 277/300 final strong-depth states are correct according to its recorded percentages. If reproduced, that indicates a successful generalizing optimization regime under the same architecture and budget.

Why caution is required. The STRM seed range is 0.83–99.17%. Near-zero training loss in all STRM seeds establishes training fit, not extrapolation. The outcome may reflect a sharp optimization phase transition, or it may expose a bug in training, evaluation, or artifact handling. It is therefore not SOTA evidence and not yet a stable research contribution.

8 Verification and next steps

The persisted-artifact audit has passed. It found 12 result records, 12 matching checkpoints, and one shared D1 manifest hash (e5437b07e6aaa10d19b5c211e97ab0dcf7db21dfb657c5d42eac87fbf561893). The STRM seed-2 checkpoint is at step 10,000 with the intended width-64, four-loop configuration. A new evaluation process loaded that checkpoint and exactly reproduced its stored 99.17% final exact and 99.69% transition accuracy at held-out depths.

This validates the stored artifact, not the stability of training. Required next checks are a fresh-output rerun of STRM seed 2, deterministic-algorithm settings where feasible, negative controls using permuted operations and labels, structural holdout, and then D2/D3 studies. For D2/D3, first perform one durable calibration seed. Then use two independent GPU-pinned workers, each with isolated atomic records and a shared durable ledger; this improves wall-clock time but does not reduce total GPU-quota consumption. Long matrices should use Kaggle Version/batch execution rather than an unattended draft session.

9 Limitations

This is D1 only, a single scale/configuration, and generated CCB-Learn evaluation rather than an upstream sealed benchmark. It cannot support claims about D2, D3, arbitrary algorithmic tasks, language-model reasoning, or general SOTA. The local operational record is in `README.md`, `docs/BENCHMARK.md`, `docs/IMPLEMENTATION.md`, and `docs/RESEARCH.md`.